Demystifying PostgreSQL Collations: A Comprehensive Guide

PostgreSQL collations are a fundamental aspect of managing textual data, dictating how character strings are stored, compared, and sorted within a database. A thorough understanding of this feature is crucial for database administrators and developers to ensure data integrity, achieve accurate query results, and optimize performance, particularly in applications that handle multilingual content. This report provides a comprehensive overview of PostgreSQL collations, from their basic definitions and application levels to their profound impact on indexing and strategies for managing version inconsistencies.

1. Introduction to PostgreSQL Collations

This section establishes the foundational understanding of PostgreSQL collations, clarifying their core purpose and distinguishing them from character sets and encodings.

1.1 What is Collation? Definition, Purpose, and Importance

Collation in PostgreSQL is a critical feature that establishes the rules governing how character data is stored, compared, and sorted within a database. It explicitly defines the sort order and character classification behavior of text data.[1] This capability allows for precise control over how textual information is handled, whether applied broadly across a database or granularly to individual columns or operations.[1]

The primary purpose of collation is to ensure that text data behaves as expected in various SQL operations. This includes ORDER BY clauses for sorting results, equality checks (=), and comparisons within JOIN clauses. Collations are particularly vital in applications that support multiple languages, as different languages adhere to distinct sorting conventions. For instance, the ordering of characters like 'o' and 'ö' can vary significantly between German and Turkish linguistic rules.[4] By defining these rules, collations ensure that data is presented and processed in a linguistically appropriate manner, aligning with user expectations.

Understanding collation is paramount for predictable database behavior, accurate query results, and maintaining data integrity. If the collation is misconfigured, "alphabetical" sorting might not match a user's linguistic expectations, potentially leading to incorrect search results, flawed reports, or even data integrity issues if unique constraints rely on specific linguistic comparisons.[3] Collations are not an inherent part of PostgreSQL's core engine; rather, they are fundamental components of computer systems, typically provided by the underlying operating system's locale settings (such as glibc on Linux) or specialized third-party libraries like International Components for Unicode (ICU).[4]

1.2 Collation vs. Character Set and Encoding (LC_COLLATE, LC_CTYPE)

To fully grasp collations, it is essential to differentiate them from character sets, often referred to as encodings. A **character set** defines *how* text characters are stored in a database by

mapping them to specific byte sequences. Examples include UTF-8 (a variable-width encoding supporting a vast range of Unicode characters) or LATIN1 (a single-byte encoding for Western European languages).[7] Character sets dictate the repertoire of characters that can be represented and their storage requirements. PostgreSQL typically defaults to UTF-8 and generally does not support multiple character sets within a single database.[7] Conversely, a **collation** defines *how* that stored text is compared and sorted within a specific character set. It answers questions like whether 'A' comes before 'B', or if an accented character like 'é' is treated the same as 'e'.[4]

The behavior of collation is primarily governed by two specific locale categories: LC_COLLATE and LC_CTYPE. LC_COLLATE specifically controls the string sort order, determining the sequence in which characters and strings are arranged.[1] LC_CTYPE manages character classification, which involves defining what constitutes a letter, how characters are categorized (e.g., as uppercase or lowercase), and their equivalence for operations like case-insensitive comparisons.[1] These two categories are distinct from other locale settings such as LC_MESSAGES (for system messages) or LC_MONETARY (for currency formatting).[5]

A critical restriction in PostgreSQL is that a database's character set must be compatible with its LC_CTYPE and LC_COLLATE locale settings.[8] For C or POSIX locales, any character set is permissible. However, for most other libc-provided locales, only one specific character set will function correctly. This ensures that the rules for character sorting and classification align with how characters are encoded. For instance, using SQL_ASCII encoding is generally inadvisable for any non-ASCII data because it interprets byte values 128-255 as uninterpreted characters, offering no conversion or validation. This can lead to silent data corruption and unpredictable behavior when collation rules are applied.[8]

The compatibility requirement between a database's character set and its LC_CTYPE/LC_COLLATE settings reveals a fundamental design principle: these configurations are not independent but are deeply intertwined. The rules for ordering and classification (collation) fundamentally depend on how characters are represented (encoding). If these are mismatched, PostgreSQL cannot reliably interpret or sort the data, potentially leading to silent data corruption or unexpected query results. For example, if a fr_FR collation expects ISO-8859-1 encoding for 'é' but the database is UTF-8, the collation rules might misinterpret the UTF-8 byte sequence for 'é', leading to incorrect sorting. The warning against SQL_ASCII further underscores this: by bypassing encoding validation, it allows inconsistent byte sequences to be stored, creating a potential for insidious data integrity issues down the line. This highlights that the initial CREATE DATABASE command is a critical architectural decision. Developers must select an encoding (preferably UTF-8 for modern applications) and a compatible locale that will serve the long-term needs of the application, as LC_COLLATE and LC_CTYPE are fixed once the database is created.

Furthermore, LC_COLLATE and LC_CTYPE are immutable after database creation because they directly affect the sort order of indexes. Changing them post-creation would corrupt existing indexes on text columns.[8] This design decision underscores PostgreSQL's strong emphasis on data integrity and index consistency. B-tree indexes, which are the default in PostgreSQL [11], are built on the principle of ordered keys. The physical arrangement of data

within the index (e.g., which item comes before another) is determined by the collation rules at the time of index creation. If these rules change, the index's internal tree structure becomes logically invalid, leading to incorrect query results or data access errors. This necessitates a REINDEX operation to rebuild the index based on the new rules. Consequently, the default database collation choice is a foundational architectural decision, not a minor detail. If an application requires different sorting rules for different datasets, relying solely on the database default is insufficient; more granular column-level or expression-level collations become necessary to override this fixed database-level "contract" for specific data.
The following table clarifies the different categories of locale settings in PostgreSQL, highlighting LC_COLLATE and LC_CTYPE as the core components of collation. This helps to understand that "locale" is a broader concept than just collation.

Table: Collation vs. Character Set & Locale Categories

Locale Category	Description
LC_COLLATE	Defines the string sorting order.
LC_CTYPE	Defines character classification (e.g., what is a letter, case equivalence).
LC_MESSAGES	Language of messages.
LC_MONETARY	Currency format.
LC_NUMERIC	Number format.
LC_TIME	Date and time format.

2. Collation Levels and Application

PostgreSQL offers fine-grained control over collation application, allowing rules to be defined at the database, column, or even expression level. This section details each level and provides practical SQL syntax examples.

2.1 Database-Level Collations

Database-level collation sets a consistent foundation for text handling across the entire database. It determines how string data is sorted and compared by default for all text data within that database, unless explicitly overridden at a lower level.[1] This approach simplifies configuration and helps maintain consistency for general text handling.
These settings are typically specified using the LC_COLLATE and LC_CTYPE clauses when creating a new database. For example, to create a database named zion that uses US English rules for character comparison with UTF-8 encoding, the syntax would be:
CREATE DATABASE zion LC_COLLATE = 'en_US.UTF-8' LC_CTYPE = 'en_US.UTF-8';.1 The default collation for a database directly reflects the LC_COLLATE and LC_CTYPE values set at its creation.16
It is important to note a limitation: PostgreSQL's database-level collation is quite restricted and does not directly support non-deterministic collations, such as case-insensitive ones.[18] This means that while it provides a broad default, it often requires more granular settings for nuanced text handling.

The database-level collation provides a consistent default, which simplifies initial setup. However, its limitations, particularly regarding non-deterministic collations, mean it is often insufficient for complex, real-world applications requiring nuanced text handling (e.g., true case-insensitivity across the board). This suggests that while it offers a convenient starting point, developers cannot simply rely on the database default for all string operations. They must be prepared to override or augment it at lower levels (column, expression) or use custom collations for advanced features, effectively pushing the responsibility for fine-grained text handling down to the application or column level.

2.2 Column-Level Collations

Column-level collations provide more granular control, allowing database designers to define specific sorting and comparison rules for individual columns within a table. When a column-level collation is defined, it actively overrides the database-level collation for that particular column, enabling tailored sorting rules based on the column's content and specific needs.[1] This is particularly useful when a database contains heterogeneous textual data that requires different linguistic treatments.

Collations are assigned to columns using the COLLATE clause during table creation (CREATE TABLE) or modification (ALTER TABLE).

- **Example during CREATE TABLE:** To create a novels table where the name column uses a French case-insensitive collation (fr_FR.CI) for sorting and comparison, the syntax would be: CREATE TABLE novels (product_id INT PRIMARY KEY, name VARCHAR(50) COLLATE "fr_FR.CI", price DECIMAL, description TEXT);.[1] This establishes structured text handling rules for the column from its inception.
- **Example during ALTER TABLE:** To modify an existing name column in the novels table to use a German collation (de_DE), the syntax is: ALTER TABLE novels MODIFY COLUMN name VARCHAR(50) COLLATE "de_DE";.[1] A more explicit form would be ALTER TABLE novels ALTER COLUMN name TYPE VARCHAR(50) COLLATE "de_DE";.[20]

It is important to understand that changing a column's collation via ALTER TABLE ALTER COLUMN TYPE can be a resource-intensive and potentially blocking operation for large tables. This process often requires the entire table and its associated indexes to be rewritten, which can temporarily consume up to double the disk space.[20] Furthermore, indexes on the modified column must always be rebuilt after a collation change to ensure their integrity and correct ordering based on the new collation rules.[20]

The ability to define column-level collations is a key feature for managing heterogeneous data. A single database might contain data from multiple regions or languages within different columns (e.g., product names in French, customer addresses in German). The database-level collation might not be appropriate for all such columns. Column-level collation provides the necessary flexibility to apply language-specific sorting and comparison rules where needed, overriding the general database default. This means that column-level collations are a crucial tool for building truly internationalized applications that need to respect diverse linguistic conventions for different data types. This approach can also optimize performance by narrowing the scope of complex collation rules to only the columns that require them.

2.3 Expression-Level Collations

Expression-level collation offers the most dynamic and granular control over text handling. It allows developers to specify the collation for individual string expressions directly within their SQL queries, providing a temporary override to the default collation of the column or database for that specific operation without altering the underlying column definition.[1]
The COLLATE clause is used inline within a string expression.

- **Example in a SELECT statement:** To apply the en_US collation to the name column for a specific SELECT operation, the query would be: SELECT name COLLATE "en_US" FROM customers;.[1]
- **Example in an ORDER BY clause:** To sort a book_title column case-insensitively using US English rules, the query would be: SELECT book_title COLLATE "en_US.CI" FROM books ORDER BY book_title;.[1]

When multiple collations are involved in a function or operator within a query, PostgreSQL follows specific rules for collation combination. Explicit collation derivations (those specified with COLLATE) take precedence. If multiple input expressions have explicit collation derivations, all of them must be the same; otherwise, PostgreSQL will raise an error. If there are conflicting non-default implicit collations (i.e., collations derived from column definitions without explicit COLLATE clauses), the result is deemed to have an indeterminate collation. This state is not an error unless the particular function being invoked requires a known collation, in which case it will lead to an error.[17]
Expression-level collations are valuable for handling ad-hoc linguistic requirements and optimizing specific queries. Sometimes, a specific query needs a different sorting or comparison rule than the column's default or the database's default. Expression-level collation provides this on-the-fly flexibility without altering the schema.[2] This can be particularly useful for ad-hoc reporting, temporary data analysis, or specific application features where a particular linguistic sort order is needed for a single operation. However, this flexibility comes with a trade-off: the research indicates that specifying collation per-query "could affect performance based on the size of the dataset and if the collated data exists in an index".[4] This suggests that while powerful, overuse or misapplication of expression-level collations on large datasets without corresponding indexes can lead to significant performance penalties, as the database might not be able to use existing indexes optimized for a different collation. For example, if a column is indexed with en_US collation, but a query uses COLLATE "de_DE" on that column in its ORDER BY clause, the index cannot be used because its internal ordering is based on en_US rules, leading to a full table scan and external sort. This highlights that expression-level collation is a powerful tool but requires awareness of its performance implications.

2.4 Common Collation Options

Collation options are attributes that define the specific rules for comparing and sorting strings, considering factors such as case sensitivity, accent sensitivity, and character width. These options act as a set of rules that determine the precise order of characters within a

given locale.[1] They allow for fine-tuning collation behavior beyond just the general linguistic rules of a locale.
The following table outlines commonly used collation options:

Table: Common Collation Options

Option	Syntax	Description
Case-sensitive	_CS	Uppercase and lowercase letters are treated differently. For example, "Castle" sorts before "castle".
Case-insensitive	_CI	Lowercase and uppercase letters are treated as identical. For example, "Ban" and "ban" are considered equal.
Accent-sensitive	_AS	Accented and non-accented letters are differentiated. For example, 'a' and 'ấ' are treated separately.
Accent-insensitive	_AI	Accented and non-accented letters are treated identically. For example, 'a' and 'ấ' are considered equal.
Width-sensitive	_WS	Differentiates between full-width and half-width characters. If not specified, width-insensitivity is applied by default.

The granularity of these linguistic nuances is a key aspect of PostgreSQL's collation capabilities. Languages often have various linguistic nuances beyond simple alphabetical order, such as specific rules for case, accents, or character width. PostgreSQL provides specific options (_CS, _CI, _AS, _AI, _WS) to control these nuances.[1] This level of detail allows database designers to precisely match the application's requirements for text comparison and sorting, ensuring data integrity and user experience are aligned with linguistic expectations. This means that a "correct" collation isn't just about the language (e.g., French), but also about the specific *behavior* desired within that language (e.g., case-insensitive French). This necessitates careful consideration of these options during schema design, as choosing the wrong sensitivity can lead to incorrect search results or data duplication.
The following table summarizes how collations are applied at different levels within PostgreSQL, complete with illustrative SQL syntax examples for each. This provides a quick, practical reference for understanding the application methods.

Table: Collation Levels and SQL Syntax

Level	Description	SQL Syntax (CREATE)	SQL Syntax (ALTER)
Database-Level	Sets consistent rules for the entire database.	CREATE DATABASE db_name LC_COLLATE = 'locale' LC_CTYPE = 'locale'; [1]	N/A (LC_COLLATE/LC_CTYPE cannot be changed after creation) [9]
Column-Level	Overrides database collation for a specific column.	CREATE TABLE tbl (col VARCHAR(50) COLLATE "locale"); [1]	ALTER TABLE tbl ALTER COLUMN col TYPE VARCHAR(50) COLLATE "locale"; [1]
Expression-Level	Applies collation for a single query operation.	N/A	SELECT col COLLATE "locale" FROM tbl; [1], ORDER BY col COLLATE "locale"; [1]

3. Types of Collations and Providers

PostgreSQL leverages external providers for collation services, offering different types of collations with varying characteristics. This section explores the standard built-in collations, locale-specific collations provided by libc, and advanced options available through the ICU provider.

3.1 Standard Collations (C, POSIX, default)

PostgreSQL offers several standard collations that are universally available across all platforms. These include default, C, and POSIX.[16]

- The default collation for a database selects the LC_COLLATE and LC_CTYPE values that were specified at the time of database creation.[16]
- The C and POSIX collations enforce "traditional C" behavior. In this mode, sorting is performed strictly based on character code byte values, and only ASCII letters from 'A' through 'Z' are treated as letters.[16] Explicitly using COLLATE "C" in a query tells the database to ignore linguistic collation rules and instead use byte order for text comparisons.[22] These collations are known for their efficiency and stability across different PostgreSQL versions.[17]
- For UTF8 encoding, ucs_basic is an SQL standard collation name that is equivalent to C, sorting by Unicode code point values.[16]
- It is worth noting that C and C.UTF-8 are slightly different. C.UTF-8 is an enhanced version of C for UTF-8 encoding that expands the set of characters considered "letters" beyond basic ASCII and can recognize invalid Unicode code points, sorting them differently than C.[9]
- Crucially, default, C, and POSIX collations can be used regardless of the database's encoding.[16]

The C collation serves as the performance baseline and a universal fallback. Its byte-wise sorting is simpler and faster than complex linguistic rules.[16] This makes C collation the most

performant option for string comparisons and sorting when linguistic correctness is not paramount. It is always available and encoding-agnostic.[16] This suggests that C collation should be the default choice unless there is a strong, explicit requirement for locale-specific sorting. It acts as a universal, predictable, and high-performance fallback, particularly critical for internal identifiers or technical data where byte-wise order is sufficient. This also explains why text_pattern_ops indexes effectively use C collation behavior for LIKE queries.

3.2 Locale-Specific Collations (libc provider)

PostgreSQL does not provide its own default collation implementation but rather relies on external collation providers. The two primary providers are glibc (the standard C library) and ICU (International Components for Unicode), with libc being the default provider.[4]
libc collations are directly mapped to the locales installed in the underlying operating system. The initdb utility populates the pg_collation system catalog with entries for all locales available on the system during cluster initialization.[4] These collations are tied to a specific character set encoding; for example, an OS locale named de_DE.utf8 would result in a collation named de_DE.utf8 for UTF8 encoding.[16] Often, a stripped-down name like de_DE is also created, which is less cumbersome and less encoding-dependent, and is generally recommended.[16]
The initial set of libc collation names is platform-dependent, directly reflecting the locales installed in the operating system, which can be listed using the locale -a command.[9]
Within any particular database, PostgreSQL primarily considers collations that use that database's encoding or those with collencoding = -1 (meaning applicable to any encoding); other entries in pg_collation are ignored.[24] If a libc collation with different LC_COLLATE and LC_CTYPE values is required, or if new locales are installed after database initialization, new collations can be created using the CREATE COLLATION command, or imported en masse using the pg_import_system_collations() function.[16]
The reliance on libc collations introduces a hidden OS dependency and its inherent fragility. Since libc collations are the default provider and directly map to OS locales [4], PostgreSQL's "linguistic intelligence" for these collations is effectively outsourced to the underlying operating system. This means that the behavior of libc collations can vary significantly between different OS versions or even different Linux distributions, even if the collation name is identical.[6] This introduces a subtle but critical fragility: an OS update or migration to a slightly different OS environment could silently alter collation behavior, potentially leading to data inconsistencies or incorrect query results without any change to the PostgreSQL version itself. This underscores the importance of maintaining consistent OS environments in production and performing careful testing during OS upgrades.

3.3 Advanced Collations (ICU provider, Deterministic vs. Non-deterministic)

The ICU (International Components for Unicode) provider offers advanced collation capabilities in PostgreSQL. ICU collations provide behavior that is independent of the operating system and database encoding, making them preferable for applications that require consistent text handling across different platforms.[4] ICU collations are highly powerful, allowing for precise rules regarding case, accents, and other textual aspects.[18] While initial

support for ICU collations was introduced in PostgreSQL 10, using an ICU collation as the default for a cluster or database became available from PostgreSQL 15.[4]
A key concept with advanced collations is the DETERMINISTIC property:

- **Deterministic (default true):** In a deterministic comparison, strings that are not byte-wise equal are considered unequal, even if they are logically considered equal by the collation rules. PostgreSQL resolves any ties by performing a byte-wise comparison.[25] This means, for example, that 'a' and 'A' might be treated as distinct if their byte representations differ, even if a case-insensitive rule is conceptually applied.
- **Non-deterministic (PostgreSQL 12+):** This setting allows the collation to behave in a truly case-insensitive or accent-insensitive manner, where logically equivalent strings (e.g., 'a' and 'A', or 'a' and 'á') are treated as identical for comparison purposes.[18] This crucial feature was not supported by older deterministic collations prior to PostgreSQL 12.[18] It is important to note that non-deterministic collations are *only* supported when using the ICU provider.[25]
- A current limitation is that it is not yet possible to use pattern matching operators such as LIKE on columns with a non-deterministic collation.[18]

ICU collations serve as a robust solution for cross-platform consistency, though they come with specific considerations. Since libc collations are OS-dependent, they can lead to potential inconsistencies across platforms or OS versions.[6] ICU collations, however, are explicitly designed to be OS-independent.[4] This makes ICU the preferred provider for applications requiring consistent text handling across diverse deployment environments. The implication is that while ICU addresses a major portability challenge, the limitation that LIKE queries cannot use non-deterministic collations [18] means there are still functional trade-offs. Developers must carefully weigh the benefits of cross-platform consistency against the limitations for LIKE and potentially plan for alternative search mechanisms or explicit COLLATE "C" for LIKE operations if non-deterministic behavior is desired for other comparisons.

3.4 Creating Custom Collations (CREATE COLLATION)

PostgreSQL allows users to define custom collations, treating them as first-class, named database objects that can be created and dropped, much like tables.[18] The CREATE COLLATION command is used for this purpose, allowing the specification of various properties for the new collation.
The general syntax for creating a custom collation is:
CREATE COLLATION name ([LOCALE = locale,])
Alternatively, a new collation can be created by copying an existing one:
CREATE COLLATION name FROM existing_collation.25
Key parameters for CREATE COLLATION include:

- name: The unique name for the new collation, which can be schema-qualified.[25]
- locale: A shortcut for setting both LC_COLLATE and LC_CTYPE simultaneously when the libc provider is used. For the builtin provider, locale must be C or C.UTF-8.[25]
- lc_collate and lc_ctype: Allow specifying individual locale categories for the libc

provider.[25]

- provider: Specifies the collation provider, which can be builtin, icu, or libc (the default).[25]
- deterministic: A boolean parameter (true by default) that controls whether the collation performs deterministic comparisons. Setting it to false enables true case- or accent-insensitivity, but this option is only available with the ICU provider.[25]
- rules: This parameter is used with the ICU provider to specify additional collation rules, allowing for highly customized sorting behavior.[25]
- version: Stores the provider-specific version of the collation, which is crucial for detecting mismatches later.[25]

Examples of custom collation creation:

- To create a collation from an operating system locale using the libc provider: CREATE COLLATION french (locale = 'fr_FR.utf8');.[25]
- To create a collation using the ICU provider for German phone book sort order: CREATE COLLATION german_phonebook (provider = icu, locale = 'de-u-co-phonebk');.[25]
- To create a custom collation with specific ICU rules: CREATE COLLATION custom (provider = icu, locale = 'und', rules = '&V << w <<< W');.[25]
- To create a collation by copying an existing one: CREATE COLLATION german FROM "de_DE";.[25]

Creating a collation requires CREATE privilege on the destination schema.[25] It also takes a SHARE ROW EXCLUSIVE lock on the pg_collation system catalog, meaning only one CREATE COLLATION command can run at a time.[25]

Custom collations are essential for application-specific logic. Standard locales or even general ICU locales might not cover highly specific application requirements, such as a custom sort order for product codes or a very particular definition of case/accent insensitivity for a domain-specific language. CREATE COLLATION with ICU and RULES allows for extremely fine-grained control over sorting and comparison.[25] This capability extends beyond generic linguistic rules to enable application-defined "business logic" for text sorting, ensuring that data ordering aligns perfectly with specific business needs or industry standards. The implication is that for complex or niche applications, custom collations become a powerful tool for data consistency and user experience, potentially reducing the need for application-side sorting logic and improving performance by pushing collation rules into the database engine.

The following table provides a concise comparison of the different collation providers, their key characteristics, and the features they support. This is crucial for understanding the implications of choosing one provider over another, especially regarding portability and advanced linguistic requirements.

Table: Collation Providers and Characteristics

Provider	Description	Characteristics	Supported Features
libc	Operating system's standard C library. Default.	OS-dependent, tied to specific encoding, can change with OS	Basic locale-specific sorting, LC_COLLATE, LC_CTYPE.

		upgrades, generally deterministic.	
ICU	International Components for Unicode library.	OS-independent, consistent across platforms, supports non-deterministic collations.	Advanced linguistic rules, DETERMINISTIC=false, RULES for custom sorting.
builtin	PostgreSQL's internal collations.	Includes C and C.UTF-8 (for UTF-8 encoding).	Byte-value sorting (C), basic Unicode code point sorting (C.UTF-8).

4. Performance Implications and Indexing

The choice of collation in PostgreSQL has significant implications for database performance, particularly concerning string operations and the efficiency of B-tree indexes.

4.1 Impact of Non-C Collations on String Operations (Sorting, Comparisons)

Using non-C collation rules can lead to a substantial degradation in performance for operations involving string comparisons. The overhead can be several to dozens of times higher compared to using the C collation.[5]

- **Sorting Operations:** Experiments demonstrate a stark difference in execution times. For instance, sorting 1.5 million Apple Store app names using zh_CN collation took approximately ten times longer (14852 ms) than using C collation (1430 ms).[5] Detailed execution plans for non-C collations often reveal "Sequential scan, external sort" operations, indicating a high computational cost and the spilling of data to disk, even when sufficient memory is available.[5]
- **Comparison Operations:** Direct string comparisons, such as those in WHERE clauses (e.g., WHERE name > 'World'), also incur significant penalties. Using zh_CN collation for 1.5 million comparison operations took nearly three times longer (441 ms) than using C collation (145 ms).[5]

The underlying reason for this performance disparity lies in the complexity of collation rules. Non-C collation rules involve intricate logic for determining string equivalence and ordering, which goes far beyond simple character code point comparisons. In contrast, the C locale's comparison rules are straightforward, comparing byte values directly (memcmp), making it significantly faster.[5]

The performance cost of linguistic correctness is a critical consideration. Non-C collations apply complex linguistic rules for sorting and comparison.[5] These rules are computationally more intensive than the simple byte-wise comparisons used by C collation.[5] This increased complexity directly translates to significantly longer execution times for operations involving string data, especially sorting and comparisons. This implies a fundamental trade-off: achieving linguistically correct sorting and comparison comes at a measurable performance

cost. Database designers must explicitly decide whether the performance overhead is acceptable for the linguistic accuracy gained, or if C collation (and its simpler, faster behavior) is sufficient for certain data types. This requires a clear understanding of application requirements versus performance expectations.

4.2 Collation and B-tree Indexes (Why LIKE queries are affected)

The choice of collation profoundly impacts the efficiency of B-tree indexes, particularly for LIKE queries. B-tree indexes are the default index type in PostgreSQL, automatically created for primary key and unique constraints. They are balanced tree structures that store keys and pointers to data rows, relying on a strict ordering of keys for efficient searching and range queries.[4] The collation used for the column dictates how values are ordered and stored within the index.[28]

A significant functional limitation arises when using non-C collations: they prevent LIKE queries from utilizing regular B-tree indexes.[5] This is because index construction relies on a predictable ordering derived from equality and comparison operations. Non-C locales introduce complex equivalence rules, such as those in the Unicode standard where multiple characters can combine to form a string equivalent to a single character. These rules are incompatible with the simpler, code-point-based ordering used by default B-tree indexes.[5] Consequently, the query planner cannot trust the index's ordering for LIKE patterns and resorts to slower full table scans.

For example, a LIKE '中国%' query on a C-based database can efficiently use an existing index, completing in approximately 2 milliseconds. However, the same query on an en_US-based database cannot use the index and performs a full table scan, degrading performance to 70 milliseconds—a 30-40 times slower execution.[5] While PostgreSQL 12 introduced non-deterministic ICU collations, allowing for case-insensitive operations, the ability to use LIKE on columns with these non-deterministic collations for index usage is still not fully supported.[18]

The indexing dilemma, balancing linguistic correctness with byte-wise order, is a core challenge. B-tree indexes rely on a consistent, predictable sort order to efficiently locate data.[11] LIKE pattern matching, especially with leading wildcards, needs to traverse the index based on the collation rules. Non-C collations introduce complex, context-dependent linguistic rules for comparison and equivalence that are fundamentally different from the simple byte-wise order used by default B-tree indexes.[5] This mismatch means the index, built on one set of ordering rules, cannot guarantee correct results for queries using a different, more complex set of rules. PostgreSQL's query planner thus avoids using the index, reverting to slower full table scans.[5] This implies that for columns requiring LIKE queries *and* linguistic collation, a standard B-tree index on the column's default collation is insufficient. This forces DBAs to choose between performance (using C collation for indexing) or linguistic correctness (accepting full table scans or using specialized indexes).

4.3 Strategies for Indexing with Non-C Collations

To mitigate the performance impact and enable LIKE queries to use indexes on columns with

non-C collations, specific indexing strategies are required:

- **Specialized Indexes:** Create indexes using text_ops COLLATE "C" or text_pattern_ops on the affected columns.[5] The text_pattern_ops operator family is specifically designed for pattern matching and performs character-by-character comparison, effectively ignoring locale rules and behaving similarly to C collation.[5]
- **Trade-offs:** These specialized indexes come with additional maintenance costs and storage space, as they might duplicate indexing that would otherwise be implicitly handled by existing PRIMARY KEY or UNIQUE constraints.[5]
- **Alternative Search Operators:** For simple case-insensitive LIKE operations, the PostgreSQL-specific ILIKE operator can be used. ILIKE performs case-insensitive pattern matching without relying on collation for case insensitivity.[18]
- An index field can also be an expression (e.g., upper(col)) to allow index usage for transformed data, though this can be complex.[28] Indexes with non-default collations can also be useful for queries that involve expressions using those non-default collations.[28]

The need for targeted solutions to mitigate the performance/indexing gap is evident. The core problem is the incompatibility between non-C collations and standard B-tree indexes for LIKE queries.[5] PostgreSQL offers text_pattern_ops and COLLATE "C" on indexes as workarounds.[5] These solutions effectively force a C-like byte-wise comparison *for the index*, allowing LIKE queries to use it. This implies that while PostgreSQL provides mechanisms to address the indexing limitation, they come with overhead (extra indexes, storage, maintenance). This means DBAs need to be proactive in identifying columns that require both linguistic collation and LIKE query performance, and then apply these specific indexing strategies. It is not a "set it and forget it" scenario, but a targeted optimization for specific use cases.

The following table provides concrete, quantitative evidence of the performance penalties associated with non-C collations, making the abstract concept of "performance degradation" tangible.

Table: Performance Impact of Collations (Example Data)

Scenario	Collation Used	Time (ms)	Notes/Impact
Sorting 1.5M records	No sort (using index)	180	Baseline for comparison.
	order by name (using index)	969	Default sort.
	order by name COLLATE "C"	1430	Sequential scan, external sort (due to explicit COLLATE, bypassing default index).
	order by name COLLATE "en_US"	10463	Sequential scan, external sort.
	order by name COLLATE "zh_CN"	14852	Sequential scan, external sort, ~10x

			slower than "C".
1.5M comparison operations (name > 'World')	Default	120	Baseline.
	C	145	Fast.
	en_US	351	Slower.
	zh_CN	441	~3x slower than "C".

5. Managing Collation Versions and Mismatches

Collation versioning is a critical, often overlooked, aspect of PostgreSQL administration. Mismatches can lead to severe data integrity issues and operational problems.

5.1 Understanding Collation Versioning

PostgreSQL does not handle collation versioning internally; instead, it outsources this responsibility to the underlying operating system libraries, primarily glibc or ICU.[4] When a collation object is created within PostgreSQL, its provider-specific version is recorded in the pg_collation system catalog.[24]

A significant issue arises when these external libraries change, for instance, due to an operating system upgrade or when using pg_upgrade to update server binaries linked with a newer version of ICU. Such changes can subtly alter the ordering rules defined by the collation. This leads to an inconsistency, or "mismatch," between the collation version recorded in the database and the version provided by the currently active library.[6]

The consequences of such a mismatch can be severe: wrong query results, duplicate data violating unique constraints, and corrupt indexes.[6] This occurs because the database system relies on stored objects (especially indexes) having a consistent sort order. If the underlying rules change, the physical ordering on disk no longer matches the logical ordering expected by the new library. Some cloud providers, like AWS RDS and Aurora, have implemented measures such as pinning glibc collation versions to mitigate this risk for their users.[6]

The fact that PostgreSQL outsources collation versioning to OS libraries creates a hidden dependency that can break database integrity. Since these libraries can update independently of PostgreSQL [6], such updates can subtly change collation rules, even if the database version remains the same.[6] PostgreSQL records the *version* of the collation at creation time.[24] A mismatch between the recorded and actual library version can lead to silent data corruption or incorrect query results, as data indexed/stored under old rules is now processed by new rules.[6] This implies that collation versioning is a critical, often overlooked, aspect of database maintenance. It transforms OS-level updates into potential database integrity issues, requiring DBAs to monitor OS library versions and proactively manage collation refreshes, rather than solely focusing on PostgreSQL version upgrades. This adds a layer of complexity to infrastructure management.

5.2 Identifying Collation Mismatches (pg_collation catalog)

To effectively manage collation versions, it is crucial to identify potential mismatches. The pg_collation system catalog is the primary resource for this, as it describes all available collations and their properties, including the recorded collversion.[24]
To inspect currently available locales and their associated collations, one can query the pg_collation catalog directly: SELECT * FROM pg_collation; or use the psql command \dOS+.[4]
To check the current collation settings of a specific database, the following queries can be used:
SELECT datname, datcollate, datctype FROM pg_database;
SHOW LC_COLLATE;
SHOW LC_CTYPE;.30
To specifically identify collations in the current database that have version mismatches and the objects (e.g., indexes, columns) that depend on them, a more advanced query can be executed:
SELECT pg_describe_object(refclassid, refobjid, refobjsubid) AS "Collation", pg_describe_object(classid, objid, objsubid) AS "Object" FROM pg_depend d JOIN pg_collation c ON refclassid = 'pg_collation'::regclass AND refobjid = c.oid WHERE c.collversion <> pg_collation_actual_version(c.oid) ORDER BY 1, 2;.29 This query leverages the pg_depend catalog to find objects linked to collations and compares their recorded version with the actual version provided by the system.
Proactive monitoring for data integrity is paramount. Collation mismatches are a silent threat that can lead to data corruption.[6] PostgreSQL provides system catalogs (pg_collation, pg_database) and functions (pg_collation_actual_version) to inspect collation details and detect mismatches.[24] This means DBAs have the tools to proactively identify potential issues *before* they manifest as critical data problems or performance regressions. The implication is that regular monitoring of collation versions, especially after OS or PostgreSQL updates, should be a standard operational practice. Relying solely on error messages or user reports of incorrect sorting is a reactive and potentially damaging approach.
The following table provides a quick reference for the pg_collation system catalog, which is essential for diagnosing and understanding collation settings and potential mismatches.
Table: pg_collation Catalog Columns (Relevant for Mismatch Identification)

Column	Type	Description
collname	name	Collation name (unique per namespace and encoding).
collprovider	char	Provider of the collation: d=database default, b=builtin, c=libc, i=icu.
collisdeterministic	bool	Indicates whether the collation is deterministic.
collencoding	int4	Encoding in which the collation is applicable, or -1 for any encoding.
collcollate	text	LC_COLLATE for libc provider

		(NULL otherwise).
collctype	text	LC_CTYPE for libc provider (NULL otherwise).
colllocale	text	Collation provider locale name for non-libc provider (NULL otherwise).
collversion	text	Provider-specific version of the collation.

5.3 Resolving Mismatches (ALTER COLLATION REFRESH VERSION, ALTER DATABASE REFRESH COLLATION VERSION)

When a collation version mismatch is detected, PostgreSQL typically issues a warning (e.g., WARNING: collation "xx-x-icu" has version mismatch).[29] The primary command to address this is ALTER COLLATION... REFRESH VERSION. This command updates the recorded collation version in the system catalog to match the current version provided by the underlying library, which makes the warning disappear.[29] For the database's default collation, an analogous command exists: ALTER DATABASE... REFRESH COLLATION VERSION.[29]

It is crucial to understand that merely refreshing the collation version is often insufficient. A change in collation definitions can lead to corrupt indexes and other problems because the database system relies on stored objects having a certain sort order. Therefore, after refreshing the collation version, all objects that depend on that collation (especially indexes) *should be rebuilt* (e.g., by using REINDEX). The REFRESH VERSION command itself does *not* verify whether all affected objects have been rebuilt correctly; it only updates the metadata in the catalog.[29]

Version information for libc collations is recorded on systems using the GNU C library (most Linux systems), FreeBSD, and Windows. For ICU collations, the version information is provided by the ICU library and is available on all platforms.[29]

The two-step recovery process is critical for ensuring data integrity after a collation mismatch. A collation mismatch is detected.[29] The REFRESH VERSION command updates the *metadata* about the collation version in PostgreSQL's catalogs.[29] However, this command *does not* re-sort or re-index the actual data on disk.[29] Since indexes (and potentially data if ordering affects storage) were built using the *old* collation rules, they are now "corrupt" or inconsistent with the *new* rules.[6] Therefore, a *second step* of rebuilding affected objects (primarily indexes, via REINDEX) is crucial to ensure data integrity and correct query behavior under the new collation rules.[29] This implies that REFRESH VERSION is a necessary but insufficient step for full recovery from a collation mismatch. DBAs must plan for the accompanying REINDEX operations, which can be resource-intensive and require downtime or careful CONCURRENTLY usage, highlighting the operational impact of these seemingly minor version changes.

5.4 Advanced Troubleshooting and Migration Strategies

In situations where refreshing the collation version and rebuilding indexes do not fully resolve

issues, or for specific migration scenarios, more advanced strategies may be necessary.

- Template Database Errors (template0, template1): Template databases are special and cannot be modified directly in the same way as regular databases. If template1 (the default template for new databases) has a collation mismatch, it can be temporarily set to allow connections, then its collation version refreshed, and finally connections disabled again:
  UPDATE pg_database SET datallowconn = true WHERE datname = 'template1';
  ALTER DATABASE template1 REFRESH COLLATION VERSION;
  UPDATE pg_database SET datallowconn = false WHERE datname = 'template1';.30
  If these steps prove insufficient, template1 can be safely dropped and recreated from template0 (the pristine template database):
  DROP DATABASE template1; CREATE DATABASE template1 WITH TEMPLATE = template0 OWNER = postgres;.30
- **Full Database Rebuild:** If persistent collation issues remain, a robust solution is to perform a full database dump, drop the existing database, recreate it with explicit and correct LC_COLLATE and LC_CTYPE settings (using template0), and then restore the data. This process ensures that all database objects are aligned with the operating system's collation version.[30]
  1. **Dump the database:** pg_dumpall -U postgres > backup.sql.[30]
  2. **Drop and recreate the database:** DROP DATABASE mydatabase; CREATE DATABASE mydatabase LC_COLLATE 'en_US.UTF-8' LC_CTYPE 'en_US.UTF-8' TEMPLATE template0;.[30]
  3. **Restore the database:** psql -U postgres -d mydatabase -f backup.sql.[30]
- **Migration Considerations:**
  - When migrating PostgreSQL instances, particularly across different operating system versions or distributions, maintaining collation consistency is vital. PostgreSQL expects the same version and OS for streaming replication to avoid issues.[6]
  - Cloud providers like AWS RDS and Aurora have introduced measures such as pinning glibc collation versions to mitigate this issue for their users.[6]
  - Migrating from other database systems, such as SQL Server, to PostgreSQL introduces specific collation challenges. SQL Server is case-insensitive by default for text comparisons, whereas PostgreSQL is case-sensitive. This fundamental difference necessitates careful collation management in PostgreSQL, potentially requiring the use of specific collations, the citext extension, or the ILIKE operator to replicate the desired case-insensitive behavior.[18]

The "nuclear option" of a full database rebuild and the complexities of cross-platform migration highlight critical aspects of collation management. Simple REFRESH VERSION and REINDEX might not always resolve complex collation issues, especially in template databases or after major environmental changes.[30] The "dump, drop, recreate, restore" strategy is presented as a guaranteed way to align all database objects with the desired collation.[30] This is effectively a "nuclear option" that ensures a clean slate for collation. The implication is that

while powerful, this method involves significant downtime and operational complexity, reinforcing the need for proactive collation management to avoid such drastic measures. Furthermore, cross-platform migrations (e.g., SQL Server to PostgreSQL) introduce additional collation-related challenges due to differing default behaviors (case sensitivity), requiring explicit planning and potentially custom collations or alternative data types (citext) to maintain application logic.

6. Best Practices and Common Pitfalls

Adopting best practices for collation management is crucial for building robust, performant, and maintainable PostgreSQL applications, especially those handling multilingual data.

6.1 When to Use C Collation

The C collation stands out as a high-performance and reliable choice for many scenarios in PostgreSQL.

- **General Recommendation:** It is widely recommended to always use UTF8 character encoding and C collation rules unless there is a strong, explicitly demonstrated need for linguistic sorting.[5]
- **Performance Benefits:** C collation offers significantly better performance for string comparisons and sorting, often being several to dozens of times faster than locale-specific collations.[5] This is due to its simple byte-by-byte comparison logic.[22]
- **Indexing Advantage:** Crucially, C collation allows LIKE queries to utilize regular B-tree indexes, preventing costly full table scans.[5]
- **Stability:** Its behavior is stable and predictable across different PostgreSQL versions and encodings.[17]
- **Use Cases:** It is ideal for internal identifiers, technical data, or any scenario where linguistic sorting is not critical, and raw performance is paramount.[22]

The C collation can be considered the "performance default" for most use cases. The evidence consistently highlights the performance benefits of C collation and the severe penalties of non-C collations, especially for indexing with LIKE.[5] This suggests that C collation should be the default choice for most text columns unless a specific linguistic sorting rule is *absolutely required*. The implication is that defaulting to OS-provided non-C collations (which is common) is a common pitfall that can lead to hidden performance bottlenecks and functional limitations. A conscious decision to use C collation for performance-critical text fields, or for data where linguistic ordering is irrelevant, is a key best practice for robust PostgreSQL applications.

6.2 Avoiding SQL_ASCII and char(n)

Certain data type and encoding choices can inadvertently undermine collation effectiveness and introduce subtle data integrity issues.

- **SQL_ASCII Encoding:**
  - **Pitfall:** SQL_ASCII does not enforce any particular encoding and performs no character validation or conversion. This means it treats all data as raw bytes,

which can lead to locale-dependent misbehavior and silent data corruption if non-ASCII characters are stored.[8] PostgreSQL will be unable to help with converting or validating non-ASCII characters.[8]
    - **Recommendation:** It is strongly advised to avoid SQL_ASCII for any non-ASCII data. UTF-8 is the universally recommended encoding for modern applications due to its comprehensive character support.[8]
- **char(n) Data Type:**
    - **Pitfall:** The char(n) data type implicitly pads shorter values with spaces to reach the specified length. Crucially, trailing spaces are treated as semantically insignificant and disregarded during comparisons. This can lead to unexpected results if whitespace is important in collation rules or if strict length adherence is expected.[8]
    - **Recommendation:** Use TEXT instead of char(n) for variable-length strings. If a fixed length is genuinely required, use TEXT with a CHECK constraint (e.g., CHECK(length(VALUE)=3)) to enforce the desired length without the problematic padding and comparison behavior of char(n).[10] VARCHAR(n) is also generally preferred over char(n).

The choice of data type is effectively a collation prerequisite. SQL_ASCII and char(n) are problematic data types for text handling.[8] SQL_ASCII bypasses encoding validation, making collation rules unreliable. char(n)'s whitespace handling interferes with collation comparisons. These data type choices can silently undermine the effectiveness and predictability of any chosen collation, leading to subtle data integrity issues that are hard to debug. This implies that correct collation usage starts with fundamental data type selection. Using UTF-8 encoding for databases and TEXT for string columns is a foundational best practice that enables collations to function as intended, preventing a class of "hidden" text handling problems.

6.3 General Recommendations for Multilingual Applications

For applications that handle diverse linguistic data, a strategic approach to collation design is essential:

- **Explicit Collations:** Always explicitly specify collations when creating databases, tables, and columns, rather than relying solely on system defaults. This reduces ambiguity and ensures consistent, predictable behavior across environments.[15]
- **ICU for Portability and Advanced Features:** For applications requiring consistent behavior across different operating systems or complex linguistic rules (e.g., non-deterministic comparisons), favor the ICU collation provider over libc.[4]
- **Understand Performance Trade-offs:** Be acutely aware of the performance trade-offs between linguistic correctness (non-C collations) and raw performance (C collation). Design decisions should balance user experience with application responsiveness.[5]
- **Indexing for LIKE Queries:** If using non-C collations on columns frequently queried with LIKE, proactively plan for specialized indexes using text_pattern_ops or COLLATE "C" to maintain query performance.[5]

- **Collation Version Management:** Implement proactive monitoring and refresh procedures for collation versions, especially after operating system or PostgreSQL upgrades. This mitigates the risk of data corruption and ensures consistency with underlying libraries.[6]
- **Consider citext or ILIKE:** For simple case-insensitivity without the full complexity of collations, the citext extension (which provides a case-insensitive text data type) or the ILIKE operator can be useful alternatives.[18]
- **Strategic Collation Design for Future-Proofing:** Multilingual applications inherently deal with diverse text handling requirements. Relying on implicit defaults or simple configurations can lead to unexpected behavior, performance issues, or data integrity problems down the line. A strategic approach involves explicitly defining collations to reduce ambiguity, choosing ICU for portability to mitigate OS-dependent behavior, understanding performance trade-offs to enable informed decisions between linguistic correctness and speed, and proactive version management to address the external dependency risk. This implies that a well-designed collation strategy is not just about current needs but also about future scalability, maintainability, and portability. It requires a deep understanding of PostgreSQL's collation mechanisms and their interactions with the OS, indexes, and application logic to build robust and predictable multilingual systems.

7. Conclusion

PostgreSQL collations are a powerful yet complex feature, fundamental to effective text handling within a database. They provide flexible control over how string data is sorted and compared, with options available at the database, column, and expression levels. The choice of collation provider (libc vs. ICU) and the DETERMINISTIC property significantly influence behavior, particularly for case and accent sensitivity, and have profound implications for cross-platform consistency and data integrity.

A critical takeaway is the inherent trade-off between linguistic correctness and raw performance. While locale-specific collations offer precise linguistic sorting, they often incur substantial performance penalties for string operations and, crucially, prevent LIKE queries from efficiently using standard B-tree indexes. The C collation, with its simple byte-wise comparison, emerges as the performance champion and a reliable default for many use cases, especially where linguistic nuances are not paramount.

Furthermore, the external dependency of PostgreSQL's collation versioning on operating system libraries introduces a significant operational risk. Unmanaged OS upgrades can lead to collation mismatches, resulting in corrupt indexes, incorrect query results, and data inconsistencies. Proactive monitoring, coupled with ALTER COLLATION REFRESH VERSION and REINDEX operations, is essential for mitigating these risks. In severe cases, a full database rebuild may be necessary to restore consistency.

In conclusion, mastering PostgreSQL collations requires careful planning and explicit configuration from the outset of a project. Database architects and developers must make informed decisions about encoding, collation levels, and provider choices, balancing linguistic

requirements with performance goals and operational realities. By adhering to best practices, such as favoring UTF-8 encoding, understanding the performance characteristics of different collations, and diligently managing collation versions, one can build robust, predictable, and performant PostgreSQL applications capable of handling diverse textual data effectively. Consulting the official PostgreSQL documentation remains an invaluable resource for deeper dives into specific advanced topics and for staying abreast of new features and best practices.

Works cited

1. Collations - PostgreSQL - Codecademy, accessed on June 6, 2025, https://www.codecademy.com/resources/docs/postgresql/collations
2. Boost Your Query Accuracy with PostgreSQL Collation Expressions ..., accessed on June 6, 2025, https://dev.to/johnniekay/boost-your-query-accuracy-with-postgresql-collation-expressions-2060
3. Collations and string sort order in PostgreSQL - Redrock Postgres, accessed on June 6, 2025, https://www.rockdata.net/blog/collations-and-sort-order/
4. Manage collation changes in PostgreSQL on Amazon Aurora and Amazon RDS - AWS, accessed on June 6, 2025, https://aws.amazon.com/blogs/database/manage-collation-changes-in-postgresql-on-amazon-aurora-and-amazon-rds/
5. Collation in PostgreSQL | PIGSTY, accessed on June 6, 2025, https://pigsty.io/blog/admin/collate/
6. Collations in Postgres - pganalyze, accessed on June 6, 2025, https://pganalyze.com/blog/5mins-postgres-collations
7. MySQL vs. PostgreSQL: Character Sets and Collations - Simple Talk - Redgate Software, accessed on June 6, 2025, https://www.red-gate.com/simple-talk/databases/mysql/mysql-vs-postgresql-character-sets-and-collations/
8. Documentation: 17: 23.3. Character Set Support - PostgreSQL, accessed on June 6, 2025, https://www.postgresql.org/docs/current/multibyte.html
9. Documentation: 17: 23.1. Locale Support - PostgreSQL, accessed on June 6, 2025, https://www.postgresql.org/docs/current/locale.html
10. Don't Do This - PostgreSQL wiki, accessed on June 6, 2025, https://wiki.postgresql.org/wiki/Don't_Do_This
11. PostgreSQL's B-Tree index optimizations - Fujitsu Enterprise Postgres, accessed on June 6, 2025, https://www.postgresql.fastware.com/pzone/2025-02-postgresql-btree-index-optimizations
12. Documentation: 16: 67.4. Implementation - PostgreSQL, accessed on June 6, 2025, https://www.postgresql.org/docs/16/btree-implementation.html
13. PostgreSQL CREATE DATABASE - DataCamp, accessed on June 6, 2025, https://www.datacamp.com/doc/postgresql/create-database
14. PostgreSQL CREATE DATABASE Statement - Neon, accessed on June 6, 2025,

https://neon.com/postgresql/postgresql-administration/postgresql-create-database
15. PostgreSQL: create database with UTF8 encoding same as in MySQL (including character set, encoding, and lc_type) - Stack Overflow, accessed on June 6, 2025, https://stackoverflow.com/questions/9961795/postgresql-create-database-with-utf8-encoding-same-as-in-mysql-including-chara
16. PostgreSQL: The World's Most Advanced Open Source Relational ..., accessed on June 6, 2025, https://access.crunchydata.com/documentation/postgresql14/latest/collation.html
17. Documentation: 17: 23.2. Collation Support - PostgreSQL, accessed on June 6, 2025, https://www.postgresql.org/docs/current/collation.html
18. Collations and Case Sensitivity | Npgsql Documentation, accessed on June 6, 2025, https://www.npgsql.org/efcore/misc/collations-and-case-sensitivity.html
19. Documentation: 17: CREATE TABLE - PostgreSQL, accessed on June 6, 2025, https://www.postgresql.org/docs/current/sql-createtable.html
20. Documentation: 17: ALTER TABLE - PostgreSQL, accessed on June 6, 2025, https://www.postgresql.org/docs/current/sql-altertable.html
21. Set or Change the Column Collation - SQL Server - Learn Microsoft, accessed on June 6, 2025, https://learn.microsoft.com/en-us/sql/relational-databases/collations/set-or-change-the-column-collation?view=sql-server-ver17
22. purpose of collate in Postgres - Stack Overflow, accessed on June 6, 2025, https://stackoverflow.com/questions/37984264/purpose-of-collate-in-postgres
23. PostgreSQL: difference between collations 'C' and 'C.UTF-8', accessed on June 6, 2025, https://dba.stackexchange.com/questions/240930/postgresql-difference-between-collations-c-and-c-utf-8
24. Documentation: 17: 51.12. pg_collation - PostgreSQL, accessed on June 6, 2025, https://www.postgresql.org/docs/current/catalog-pg-collation.html
25. Documentation: 17: CREATE COLLATION - PostgreSQL, accessed on June 6, 2025, https://www.postgresql.org/docs/current/sql-createcollation.html
26. Postgres collate example in select? - Database Administrators Stack Exchange, accessed on June 6, 2025, https://dba.stackexchange.com/questions/195502/postgres-collate-example-in-select
27. Understanding the mechanics of PostgreSQL B-Tree indexes - Fujitsu Enterprise Postgres, accessed on June 6, 2025, https://www.postgresql.fastware.com/pzone/2025-01-understanding-the-mechanics-of-postgresql-b-tree-indexes
28. Documentation: 17: CREATE INDEX - PostgreSQL, accessed on June 6, 2025, https://www.postgresql.org/docs/current/sql-createindex.html
29. Documentation: 17: ALTER COLLATION - PostgreSQL, accessed on June 6, 2025, https://www.postgresql.org/docs/current/sql-altercollation.html
30. How to Fix PostgreSQL Collation Version Mismatch in Kali Linux ..., accessed on

June 6, 2025, https://www.webasha.com/blog/how-to-fix-postgresql-collation-version-mismatch-in-kali-linux-step-by-step-guide-to-refreshing-collation-and-resolving-common-errors
31. Rebuild and Refresh Collation Version for PostgreSQL - GitHub, accessed on June 6, 2025, https://gist.github.com/troykelly/616df024050dd50744dde4a9579e152e
32. template database "template1" has a collation version, but no actual collation version could be determined - Stack Overflow, accessed on June 6, 2025, https://stackoverflow.com/questions/75746179/template-database-template1-has-a-collation-version-but-no-actual-collation-v
33. What's the best practice for PostgreSQL database migration between on-premise servers?, accessed on June 6, 2025, https://www.reddit.com/r/PostgreSQL/comments/1itdc1n/whats_the_best_practice_for_postgresql_database/
34. Moving from SQL Server to PostgreSQL for Cost Optimization – What Are the Key Considerations? - Reddit, accessed on June 6, 2025, https://www.reddit.com/r/dotnet/comments/1j9j0sz/moving_from_sql_server_to_postgresql_for_cost/
35. Don't Do This - PostgreSQL wiki, accessed on June 6, 2025, https://wiki.postgresql.org/wiki/Don%27t_Do_This